
CIS-11 Project Bubble Sort

Team Name:

Bubble Sort Sorcerers

Team Members:

Cassandra Suarez

Bryant Martinez Fossati

Ryland Tolentino

Bubble Sort

May 24, 2026

Advisor: Kasey Nguyen, PhD

Part I – Application Overview - Ryland Tolentino

The Purpose and Vision of the Project

Objectives

The Purpose and Vision of the Project

The purpose of our Bubble Sort is to replace the existing manual queue-management processes with an automated application. By introducing centralized prioritization logic, the system will eliminate operational bottlenecks, reduce human error in task assignment, and provide standard workflows for all key user roles.

Primary Business Objectives

The success of the deployment will be measured by the application's ability to achieve the following business metrics within a few weeks of production rollout.

- **Increase Optional Efficiency:** Reduce the average time a person depends manually organizing and assigning tasks from 45 minutes per day to less than 5 minutes.
- **Improve SLA Compliance:** Ensure that 98% of high-priority business items are identified and processed before their expiration window.
- **Standardize Data Auditing:** Provide supervisors with a 100% auditable history of how data was sorted, compared, and routed, meeting new compliance standards.

System & Functional Objectives

To support our business goals, the application must meet the following functional objectives upon launch.

- **Automated Ingestion & Evaluation:** Automatically ingest daily batch volumes of up to a certain amount of records and evaluate them against defined business rules without requiring manual data entry.
- **Real-Time Prioritization:** Execute sequential comparison logic to continuously bubble the highest-urgency items to the top of the user queue.
- **Role-Based Security:** Ensure strict data segmentation so that users can only view, edit, or override the specific queues tied to their assigned responsibilities.

Business Process

Business Process Definition: Automated Queue Prioritization

Context and Trigger: The process starts when the daily batch of either customer service tickets or inventory files is uploaded to the system.

Application Role: Our application gives a sequential comparison logic to organize these items. The application automatically reviews adjacent items in the queue, comparing their metric and shifts the higher-priority items forward.

User Interaction: The operations agent does not see the sorting happen, and instead, their workflow is transformed: they open the application and are immediately presented with a perfectly prioritized list, allowing them to work from top to bottom without wasting time deciding what to do next.

User Roles and Responsibilities

With the use of roles and responsibilities we must define user roles and its job titles. An employee would wear multiple hats. In this section, we are defining roles based on the business objective they are trying to achieve through our application.

To map tasks and system interactions, each role lists the specific tasks that require our application. Group these tasks by frequency/time to give the business expert a clear picture of operational cadence.

Describing Task Relationships and Hand-offs systems rarely exist in a vacuum; the output of one role is usually the input for another.

User Roles	Primary Business Objective	Main System Interaction
Data Clerk	Input and clean raw incoming data batches	Batch Upload Tool, Error Log
Supervisor	Approve finalized queues and handle exceptions	Approval Dashboard, Override Tool.

Production Rollout Considerations

Rollout Strategy

- **Deployment Method:** Date out possible outcomes from different people
- **Schedule & Timeline:** May 24 - June 11
- **Rollback strategy:** If there are errors we will identify the problems given to us in the program or use other resources to help identify those mistakes in the program to better maintain and refine it.

Data population & System Priming

- **Source Systems:** From the possible outcomes *if* people were to use the program
- **Migration Window:** Possible hour or two of testing
- **Validation Method:** The business expert would perform a row-count reconciliation between the old database and the new app/program.

Capacity and Volume Constraints

- **Daily Operational Load:** If data were available to the app. The average could be around 5,000 new client leads ingested per day, in processing 15,000 items processed during end of quarter surges.
- **System Performance Targets:** The system must complete the daily queue prioritization within 5 seconds of a user initiating the request.

Terminology

Term	Domain Context	System/Project Definition
Batch Ingestion	Data Processing	The daily event (occurring at 6:00 PM) Where the application imports raw, unorganized transaction data from the legacy database.
Priority Score	Business Logic	A calculated metric based on [Value X] multiplied by [Days Outstanding]. This score determines an item's position during the automated sorting process
Exception Flag	System State	A visual alert applied to a record when the sorting algorithm encounters corrupt or missing data, routing it automatically to the Supervisor role.
Stale Record	Business Status	Any data item that has remained un-acted upon in the sorted queue for more than 48 hours.

Part II – Functional Requirements - Bryant Martinez Fossati

Statement of Functionality

Core Sort Functions

The LC-3 Bubble Sort application accepts eight integers (0 to 100) entered one at a time by the user at the LC-3 console keyboard, sorts them in ascending order using the bubble sort algorithm, and displays the sorted results to the LC-3 console. The program is implemented entirely in LC-3 assembly language using keyboard input routines, console output routines, subroutines called with JSR/RET, a software stack for register save/restore, and ASCII-to-binary and binary-to-ASCII conversion:

- **User Input:** The program reads eight integers from the keyboard one digit at a time using TRAP x23 (GETC). Each ASCII digit character is converted to its numeric value by subtracting x30. Multi-digit numbers are assembled by multiplying the running total by ten and adding each successive digit. After the user presses Enter, the completed integer is stored into the next position of an 8-element array in LC-3 memory. This process repeats eight times.
- **Adjacent Comparison:** For each adjacent pair $A[j]$ and $A[j+1]$, both values are loaded from memory with LDR (Base+offset mode). Their difference $A[j] - A[j+1]$ is computed using 2's complement subtraction: NOT is applied to $A[j+1]$, then ADD combines the result with $A[j]$ and #1. The resulting condition codes N, Z, or P determine whether a swap is needed.
- **Conditional Swap:** If the P condition code is set ($A[j] > A[j+1]$), the two elements are swapped in memory using STR instructions with Base+offset addressing. If no swap is needed, execution falls through to the next iteration.
- **Inner Loop Control:** An inner loop counter decrements by 1 after each comparison. A BRp instruction redirects execution back to the comparison step while more adjacent pairs remain in the current pass. With a fixed array size of 8 elements, each pass performs at most 7 adjacent comparisons, ensuring the largest unsorted element bubbles to its final position.
- **Outer Loop Control:** An outer loop counter tracks completed passes and decrements after each inner loop finishes. A BRp instruction restarts the inner loop for the next pass. With $n=8$ fixed, exactly 7 passes are required. When the outer counter reaches zero, execution falls through to the subroutine call that outputs the sorted results.
- **Subroutine Calls:** The program uses JSR (Jump to Subroutine) to call two subroutines: an input subroutine that reads keyboard characters and converts them to binary integers, and an output subroutine that converts sorted integers to ASCII and prints them to the console. Each subroutine preserves caller registers on the stack before modifying them and restores them with RET.
- **Stack Operations:** A software stack is maintained using a dedicated stack pointer register. Before each subroutine modifies any shared registers, it pushes their values onto the stack using STR with a pre-decremented stack pointer. On return, the saved values are popped using LDR with a post-incremented stack pointer. This prevents subroutine calls from corrupting outer loop counters or array pointers.
- **Program Termination:** After all 7 sorting passes complete, the program calls the output subroutine (JSR OUTPUT) to display the eight sorted integers to the LC-3 console in ascending order. Each integer is converted from binary to decimal ASCII digit characters and output one character at a time using TRAP x21 (OUT). After all eight values are displayed, the program executes TRAP x25 to halt the LC-3 simulator.

LC-3 Instructions Used in This Implementation:

- **AND (opcode 0101):** Clears registers to zero for initialization (AND Rx, Rx, #0).
- **LEA (opcode 1110):** Loads effective addresses into base registers. Used to initialize the array base pointer and the stack pointer at program startup.
- **ADD (opcode 0001):** Increments and decrements loop counters; performs 2's complement subtraction to compare adjacent elements.
- **NOT (opcode 1001):** Performs bitwise complement as the first step of 2's complement subtraction when comparing adjacent elements.
- **LDR (opcode 0110):** Loads array elements from memory using Base+offset addressing during sorting. Also used in subroutine epilogues to pop saved register values from the stack.
- **STR (opcode 0111):** Writes swapped elements back to their correct memory positions using Base+offset addressing. Also used in subroutine prologues to push register values onto the stack.
- **BRn/BRp/BRz (opcode 0000):** Conditional branches implement the swap decision and both inner and outer loop control based on condition codes set by the most recent operate or load instruction.
- **JSR (opcode 0100):** Calls a subroutine by saving the return address in R7 and jumping to the subroutine's start address. Used to invoke the keyboard input subroutine and the console output subroutine from the main program.
- **RET (opcode 1100):** Returns from a subroutine by loading the program counter from R7. Used at the end of each subroutine to transfer control back to the calling program.
- **TRAP x23 (GETC):** Reads one ASCII character from the keyboard into R0 without echoing to the console. Called once per digit keystroke during the input loop to capture each decimal digit the user types.
- **TRAP x21 (OUT):** Outputs one ASCII character from R0 to the LC-3 console display. Called for each decimal digit and newline separator when displaying the eight sorted values after sorting completes.
- **TRAP x25 (HALT):** Halts the LC-3 simulator after all sorted values have been displayed to the console.

Scope

This project is delivered as a single release. The following defines what is included in the current version and what is reserved for future work:

- **Phase 1 (Current Release) - Core Sort:** A fully functional LC-3 assembly implementation of bubble sort that reads exactly 8 integers (0 to 100) from the keyboard via TRAP x23, sorts them in ascending order using an in-place bubble sort algorithm, and displays the sorted values to the LC-3 console via TRAP x21. The program includes two subroutines (keyboard input with ASCII conversion, and console output with binary-to-ASCII conversion), a software stack for register save/restore, and TRAP x25 for clean program termination.
- **Out of Scope (Future Phases):** Input range validation (confirming each entered value is within 0 to 100), descending order sorting, user-selectable array size (currently fixed at 8), and support for negative integers are not included in this release and are reserved for future enhancements.

Performance

The bubble sort algorithm operates with $O(n^2)$ time complexity. For this implementation the array size is fixed at $n=8$ elements. The following numeric performance targets apply to this LC-3 implementation:

- **Comparison count:** With $n=8$ fixed, worst-case adjacent comparisons equal $n*(n-1)/2 = 8*7/2 = 28$ comparisons across all 7 passes. The sort completes in at most 28 compare-and-conditional-swap cycles.
- **Instruction execution count:** Each comparison-and-conditional-swap cycle requires at most 12 LC-3 instructions. The input and output subroutines each require approximately 30 instructions per value. Worst-case total: $(28 * 12) + (8 * 30) + (8 * 30) = 336 + 480 =$ approximately 820 LC-3 instruction executions.
- **Execution time target:** The complete program including input, sort, and output must finish within 1,500 LC-3 instruction cycles on the simulator. This budget covers 28 sort comparisons, 8 keyboard-input cycles with ASCII conversion, 8 console-output cycles, and all subroutine call overhead.
- **Memory footprint:** Program code and constants occupy addresses x3000 through approximately x303F (at most 64 LC-3 words). The data region holds exactly 8 array words plus a 10-word stack buffer, totaling 18 data words. All regions combined remain well within the LC-3 64K-word address space. Values in the range 0-100 are well within the 16-bit signed range, so arithmetic overflow does not occur during sorting.

Usability

The following usability requirements govern all interactions with the LC-3 Bubble Sort program. A user familiar with the LC-3 simulator can load and run the program in 2 setup steps. The program then prompts for 8 keyboard inputs and automatically displays the sorted results, requiring no additional user action after the last integer is entered.

- **Keyboard input:** After the program starts, the user types each of the 8 integers at the keyboard one at a time. Each integer must be a non-negative decimal value between 0 and 100. The user presses Enter after each number to confirm it. The program reads each digit character by character using TRAP x23 and converts them from ASCII to binary before storing in the array.
- **Execution:** The user sets the PC to x3000 in the LC-3 simulator and issues a Run command. The program immediately begins reading keyboard input. The user must type 8 integers sequentially at the console; the program processes each one before prompting for the next. After the eighth integer is entered, the sort runs automatically with no further user action required.
- **Console output:** After sorting completes, the program automatically prints all eight sorted integers to the LC-3 console in ascending order using TRAP x21. Each value is converted from binary to its decimal digit characters before output. The user reads the results directly from the console display; no memory inspection is required.
- **Register state after execution:** All registers saved by the subroutine stack mechanism are restored before the program returns to the caller. Registers used by the main sort loop (R0 through R5) will reflect their final loop values after halting. Users must not rely on any specific register value after TRAP x25 executes.

Documenting Requests for Enhancements

There does come a time when the requirements for the initial release of your application are frozen. Usually, it happens after the system acceptance test which is the last chance for the users to lobby for some changes to be introduced in the upcoming release.

Currently, you need to begin maintaining the list of requested enhancements. Below is a template for tracking requests for enhancements.

Date	Enhancement	Requested by	Notes	Priority	Release No/ Status
<i>May 15, 2026</i>	<i>Descending order sort option</i>	<i>Bryant Martinez Fossati</i>	<i>Allow the user to select ascending or descending sort order before execution begins</i>	<i>Low</i>	<i>Phase 2</i>
<i>May 15, 2026</i>	<i>User-defined array size</i>	<i>Bryant Martinez Fossati</i>	<i>Allow the user to specify how many integers to sort (currently fixed at 8), up to a maximum of 20</i>	<i>Low</i>	<i>Phase 3</i>
<i>May 15, 2026</i>	<i>Input range validation</i>	<i>Bryant Martinez Fossati</i>	<i>Validate each keyboard-entered integer is within 0 to 100; re-prompt the user if an out-of-range value is entered</i>	<i>Medium</i>	<i>Phase 2</i>

Part III – Appendices - Cassandra Suarez

Appendix A - Pseudocode

The Bubble Sort algorithm sorts numbers by repeatedly comparing adjacent array elements and swapping them when necessary. The implementation validates that all integers fall within the allowed range of 0 to 100 before sorting continues. The sorting process repeats until the entire array is arranged in ascending order.

BEGIN BubbleSort

LOAD n from memory address x3100

SET outerCounter = n - 1

WHILE outerCounter > 0 DO

SET pointer = base address of array x3101

SET innerCounter = outerCounter

WHILE innerCounter > 0 DO

LOAD A[j] from memory at pointer

LOAD A[j+1] from memory at pointer + 1

IF A[j] < 0 OR A[j] > 100 THEN

DISPLAY "Invalid Input"

HALT

END IF

IF A[j+1] < 0 OR A[j+1] > 100 THEN

DISPLAY "Invalid Input"

HALT

END IF

COMPUTE difference = A[j] - A[j+1]

IF difference > 0 THEN

STORE A[j+1] into memory at pointer

STORE A[j] into memory at pointer + 1

END IF

ADVANCE pointer by 1

DECREMENT innerCounter by 1

END WHILE

DECREMENT outerCounter by 1

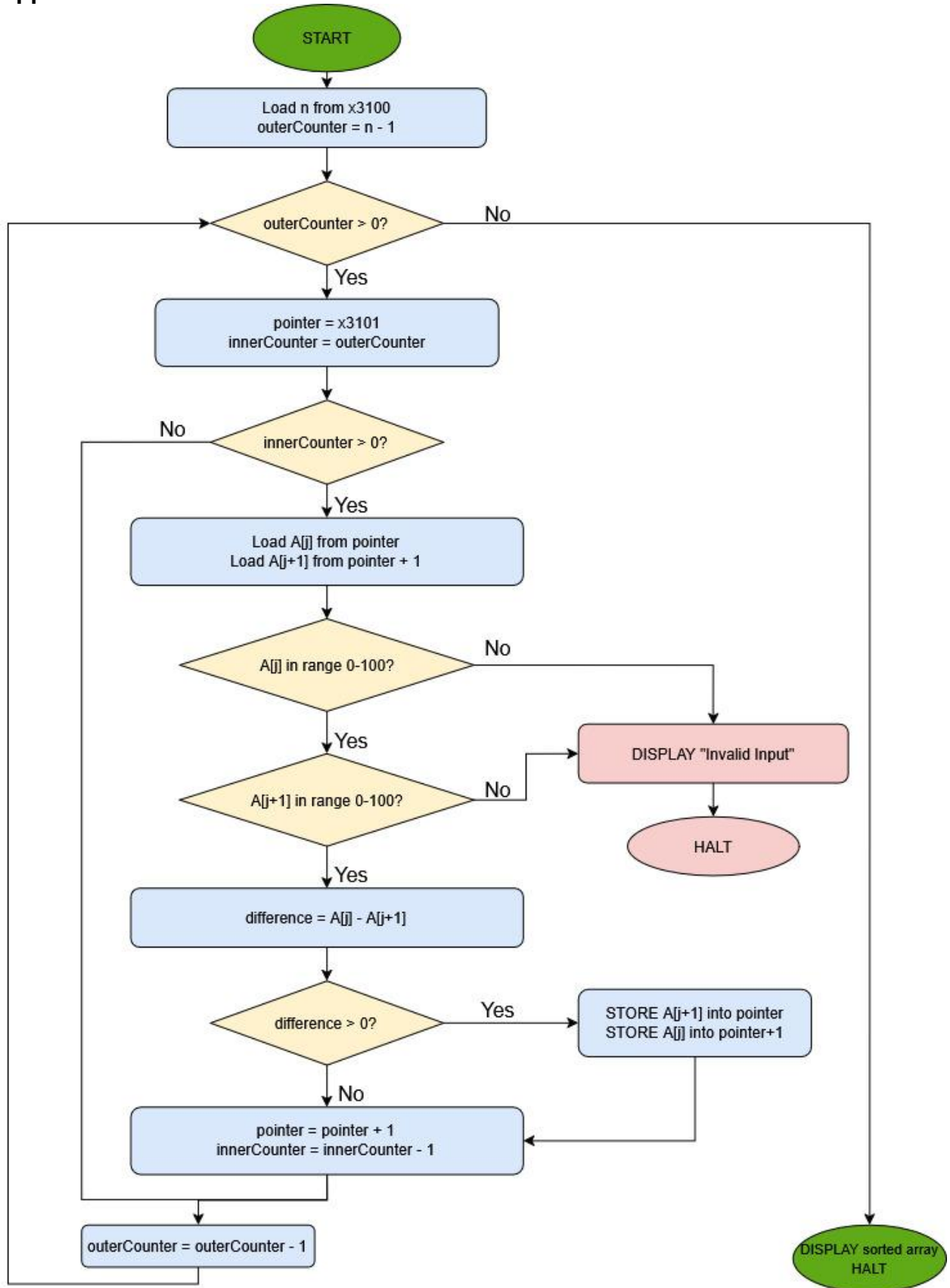
END WHILE

DISPLAY sorted array values

HALT

END BubbleSort

Appendix B - Flow Chart



Appendix C - LC-3 Instruction Reference

Instruction	Purpose in Bubble Sort Program
AND	Clears registers to zero before use
LEA	Initializes array pointer and stack pointer
ADD	Increments/decrements counters and performs arithmetic
NOT	Used for 2's complement subtraction during comparisons
LDR	Reads array values during sorting
STR	Writes swapped values back into the array
BRn	Detects invalid negative input values
BRp	Continues loops and determines swap conditions
BRz	Detects loop completion conditions
JSR	Calls input and output subroutines
RET	Returns control to the main program
TRAP x23 (GETC)	Accepts user input digits
TRAP x21 (OUT)	Prints sorted output values

TRAP x25 (HALT)	Ends the Bubble Sort program
--------------------	------------------------------

Appendix D - Register Allocation

<u>Register</u>	<u>Purpose</u>
R0	Input/output character
R1	Array pointer
R2	Outer loop counter
R3	Inner loop counter
R4	Current element
R5	Adjacent element
R6	Stack pointer
R7	Return address

Appendix E - Test Cases and Expected Results

Test #	Input	Expected Output
1	8 3 5 1 9 2 7 4	1 2 3 4 5 7 8 9
2	100 99 50 25 0 1 2 3	0 1 2 3 25 50 99 100
4	-1 5 3 2 1 0 4 6	Invalid Input

Appendix F - Metrics

Metric	Result
Algorithm	Bubble Sort
Array Size	8 integers
Worst-Case Complexity	$O(n^2)$
Best-Case Complexity	$O(n^2)$
Space Complexity	$O(1)$
Maximum Comparisons	28
Maximum Passes	7
Sorting Method	In-place adjacent comparison
Overflow Risk	None for values 0–100

